SHADERY V OPENGL

Kurz: **Moderní počítačová grafika**

Lektor: Ing. Michal Švento

Náplň cvičení

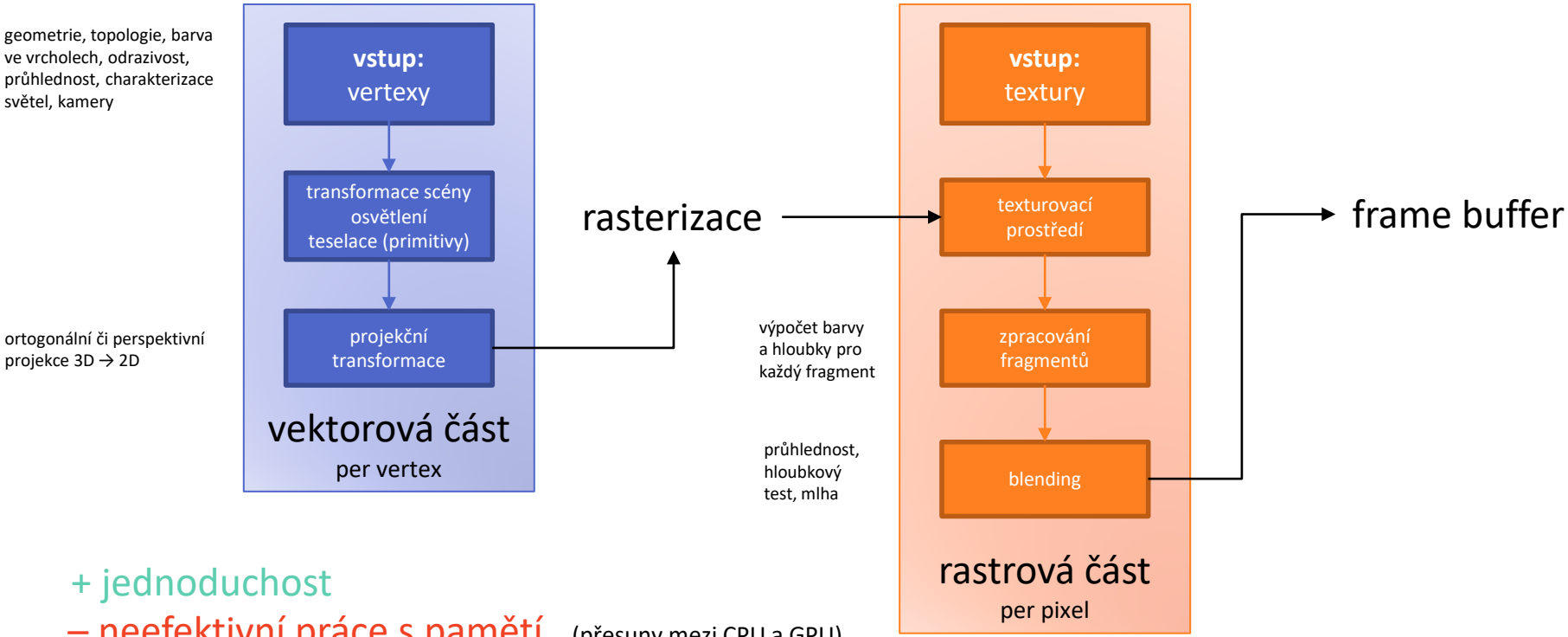
1. Vykreslovací řetězec
   - Fixní (statický)
   - Programovatelný
2. Práce s daty pomocí buffer objektů
3. Shadery v OpenGL
4. GLSL

Fixní (statický) vykreslovací řetězec

- Využití základních knihovních funkcí pro implementaci matematických operací v pozadí
  - Práce s maticemi `GL_MODELVIEW`, `GL_PROJECTION`
  - Lineární transformace: `glTranslatef`, `glRotatef`,…
- Pevně dané vertexy
  - `glVertex*`, `glColor*`, `glNormal*`,…
- Základní funkce pro osvětlování
  - `glLight*`, `glMaterial*`,…
- Texturování pomocí texturovacích prostředí

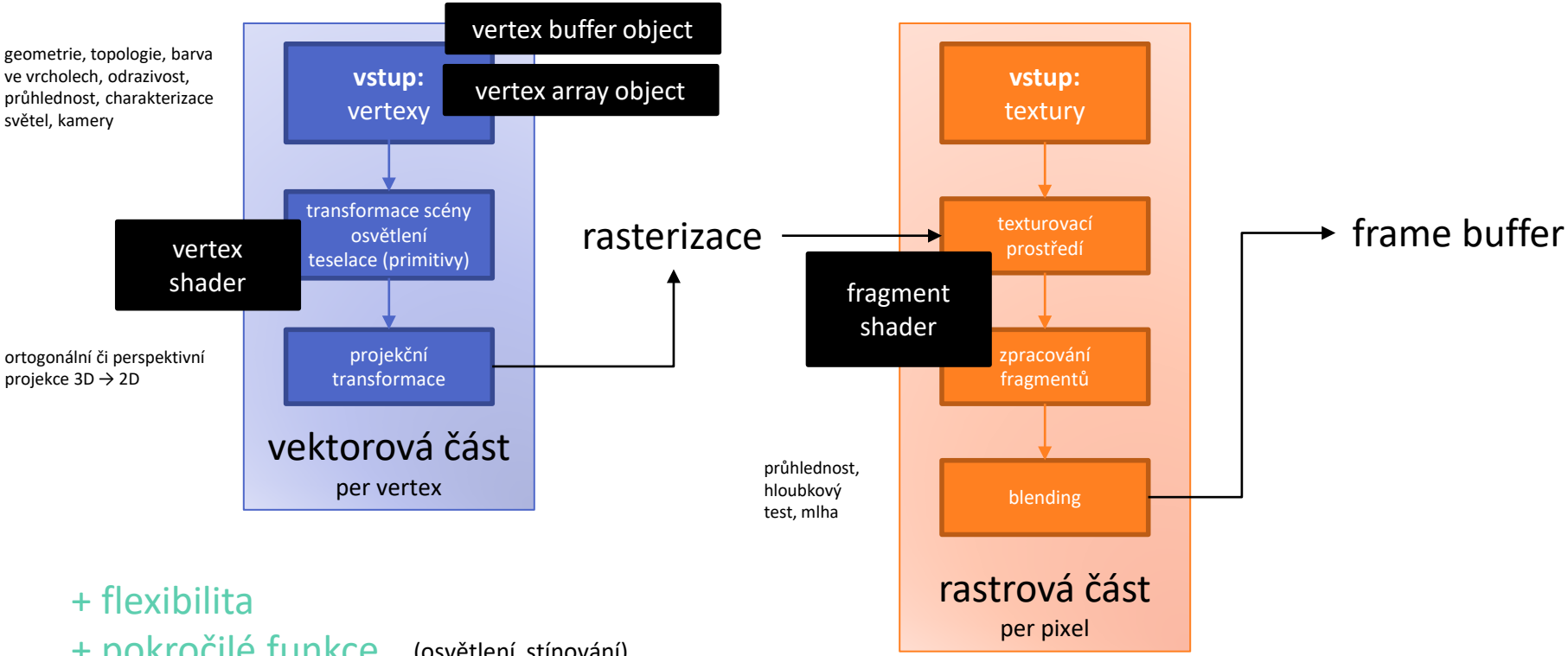
https://www.khronos.org/opengl/wiki/Legacy_OpenGL

Fixní (statický) vykreslovací řetězec



+ jednoduchost
– neefektivní práce s pamětí (přesuny mezi CPU a GPU)
– neflexibilní (základní transformace, osvětlování, texturování)

Programovatelný vykreslovací řetězec



+ flexibilita
+ pokročilé funkce (osvětlení, stínování)
– náročnější vývoj

Vertex buffer object (VBO)

Klasický přístup:

1. Vertexy jsou definovány na CPU/RAM
   - `glVertex*`, `glColor*`, `glNormal*`,…
2. Při každém vykreslení jsou přesunuty do GPU

VBO:

1. Vlastnosti vertexů (atributy) jsou definovány formou polí a navázány na příslušné buffery
   - různé vlastnosti = různé buffery lze propojit do tzv. **Vertex Array Object**
2. Data jsou po celou dobu v paměti GPU

https://antongerdelan.net/opengl/vertexbuffers.html

Vertex buffer object (VBO)

- Předpokládejme např. následující strukturu dat

```
// souradnice bodu
float points[] = {
   0.700f,  0.000f,
  -0.350f,  0.606f,
  -0.350f, -0.606f
};
```

```
// barvy bodu
float colors[] = {
  1.0f, 0.0f,  0.0f,
  0.0f, 1.0f,  0.0f,
  0.0f, 0.0f,  1.0f
};
```

- Inicializace bufferu

```
Glunit points_vbo; // identifikator (jmeno) naseho bufferu
glGenBuffers(1, &points_vbo);
```

- Navázání bufferu pro použití na atributy vertexů

```
glBindBuffer(GL_ARRAY_BUFFER, points_vbo);
```

- Propojení bufferu s daty

```
glBufferData(GL_ARRAY_BUFFER, sizeof(points), points, GL_STATIC_DRAW);
```

- Obdobně pro pole s barvami

https://antongerdelan.net/opengl/vertexbuffers.html

Vertex array object (VAO)

- Máme 2 buffery pro různé vlastnosti vertexů
  - `points_vbo`
    - navázaný na pole `points`
    - definuje 3 body pomocí 2 souřadnic typu float
  - `colors_vbo`
    - navázaný na pole `colors`
    - definuje 3 barvy pomocí 3 hodnot typu float
- Buffery provážeme pro jednodušší práci do jediného VAO

https://antongerdelan.net/opengl/vertexbuffers.html

Vertex array object (VAO)

- Inicializace VAO

```
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);
```

- Navázání bufferu se souřadnicemi vertexů

```
glBindBuffer(GL_ARRAY_BUFFER, points_vbo);
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 0, NULL);
```

krok (přeskočené hodnoty mezi po sobě následujícími vertexy)

normalizace dat

offset

velikost a typ dat

identifikátor
(bude potřeba v další fázi)

- Obdobně navázání bufferu s barvami vertexů

```
glBindBuffer(GL_ARRAY_BUFFER, colors_vbo);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, NULL);
```

Vertex array object (VAO)

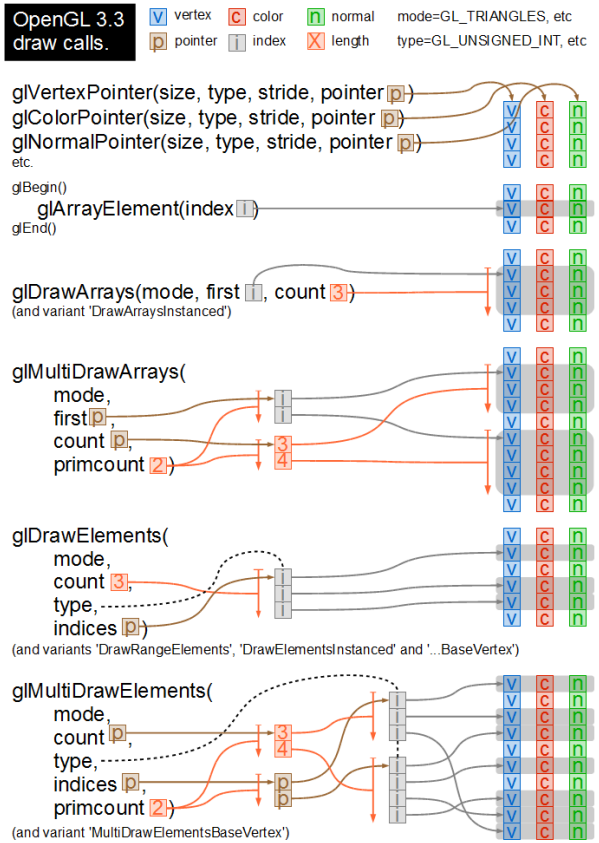
- Atributové pole bude využito při kreslení
  - `glDrawElements`
  - `glDrawRangeElements`
  - `glDrawArrays`
  - `glMultiDrawArrays`
  - `glMultiDrawElements`

- Čtení atributů musíme zapnout

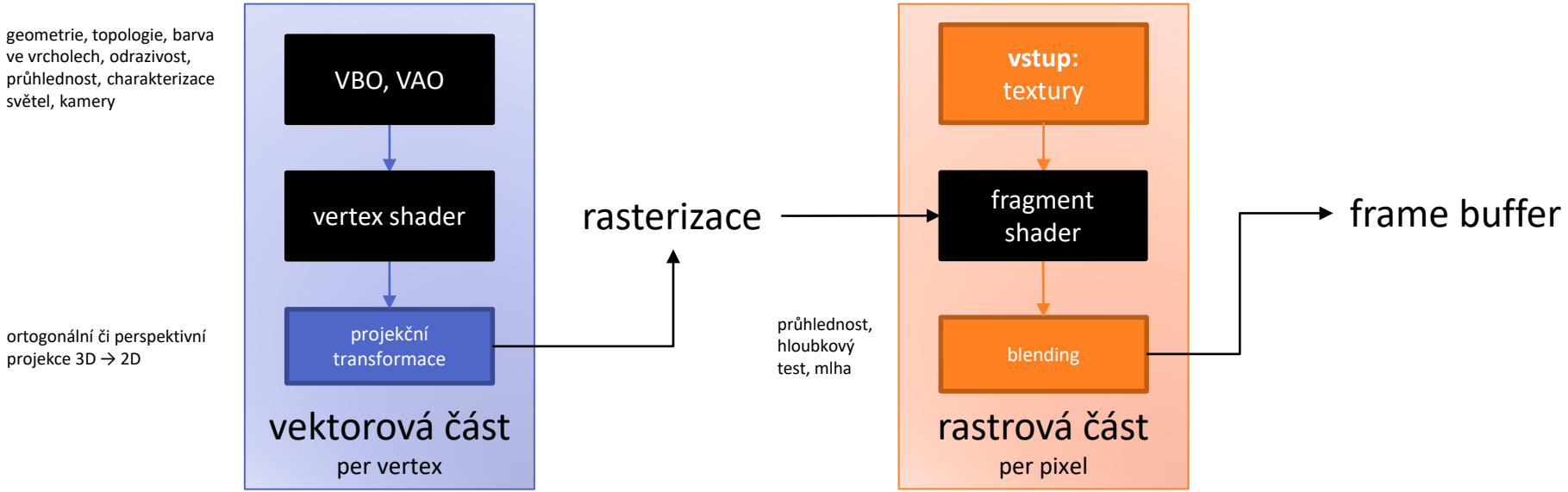
```
glEnableVertexAttribArray(0);
glEnableVertexAttribArray(1);
```

```
glBindBuffer(GL_ARRAY_BUFFER, points_vbo);
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 0, NULL);

glBindBuffer(GL_ARRAY_BUFFER, colors_vbo);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, NULL);
```



Programovatelný vykreslovací řetězec



Shader

- Malý samostatný program běžící na GPU
- Realizuje jeden článek vykreslovacího řetězce
  - transformace vertexů: **vertex shader** (vektorová část)
  - zpracování diskretizovaných fragmentů: **fragment shader** (rastrová část)
- Shadery mezi sebou komunikují pouze pomocí vstupů a výstupů

Shader

```
#version version_number
in type in_variable_name;
in type in_variable_name;

out type out_variable_name;

uniform type uniform_name;

void main()
{
    // process input(s) and do some weird graphics stuff
    ...
    // output processed stuff to output variable
    out_variable_name = weird_stuff_we_processed;
}
```

https://learnopengl.com/Getting-started/Shaders

GLSL

- OpenGL Shading Language
- Datové typy
  - skalární: `int`, `float`, `double`, `uint`, `bool`
  - vektorové: `vecn`, `bvecn`, `ivecn`, `uvecn`, `dvecn`
    - přístup k jednotlivým složkám přes `.x`, `.y`, `.z`, `.w` (nebo rgba, stpq)
    - sčítání a násobení vektorů stejné délky probíhá po složkách
  - matice: `matnxm` (n sloupců, m řádků), `matn`
    - násobení je násobení matic, ostatní operace po složkách
  - `sampler2D`
    - texturovací jednotka
- Built-in variables

Vertex shader, fragment shader

```
#version 330 core
layout (location = 0) in vec3 aPos; // the
position variable has attribute position 0

out vec4 vertexColor; // specify a color output
to the fragment shader

void main()
{
    gl_Position = vec4(aPos, 1.0);
    vertexColor = vec4(0.5, 0.0, 0.0, 1.0);
}
```

```
#version 330 core
out vec4 FragColor;

in vec4 vertexColor; // the input variable from
the vertex shader (same name and same type)

void main()
{
    FragColor = vertexColor;
}
```

vestavěné proměnné: `gl_Position`, `FragColor`

https://learnopengl.com/Getting-started/Shaders

Uniforms

- globální proměnné
- umožňují komunikaci se shaderem v průběhu programu

```
#version 330 core
out vec4 FragColor;

uniform vec4 ourColor; // we set this variable in the OpenGL code

void main()
{
    FragColor = ourColor;
}
```

https://learnopengl.com/Getting-started/Shaders

Propojení shaderů (inicializace, onInit)

- inicializace: `GLuint id = glCreateShader(GLenum type);`
  - type: `GL_VERTEX_SHADER`, `GL_FRAGMENT_SHADER`
- zdrojový kód: `glShaderSource(id, 1, &code_text, NULL);`
- kompilace: `glCompileShader(id);`

- vytvoření programu vykreslovacího řetězce:
  - inicializace: `GLuint program = glCreateProgram();`
  - připojení shaderů: `glAttachShader(program, id);`
  - linker: `glLinkProgram(program);`

Práce s proměnnými shaderů

- ve vertex shaderu máme např. následující proměnnou:

```
layout(location = 0) in vec2 vertex_position;
```

- tomu odpovídá VBO, který je provázaný s VAO prostřednictvím lokace atributu 0
- díky explicitnímu zadání lokace `layout(location = 0)` nemusíme proměnné dále propojovat

- globální proměnné uniforms

```
GLuint location = glGetUniformLocation(GLuint program, „promenna“);
```

Aplikace (běh programu, onDisplay)

- použití programu vykreslovacího řetězce: `glUseProgram(GLuint program);`
- přiřazení hodnoty uniform proměnné:

```
glUniform…(location, …, *value);
```

- např. pro matice 4x4 typu float:

```
glUniformMatrix4fv(location, 1, GL_FALSE, (const GLfloat*)matrix);
```

počet

transpozice

ukazatel na proměnnou `matrix` s daty

- vykreslení:

```
glDrawArrays
```

Úkol 1

1. Nastudujte použití shaderů ve vykreslovacím řetězci.
2. Doplňte zdrojový text pro vertex shader i pro fragment shader pro vykreslení barevného trojúhelníku.
   - Předejte vstupní proměnné do výstupních.
   - V rámci vertex shaderu musí být zadána hodnota zabudované proměnné `gl_Position` jako homogenní souřadnice (`vec4`).

Úkol 2

1. Pomocí úpravy globální uniform proměnné `mat4 MVP` realizujte otáčení trojúhelníku.
   - Ve vhodném místě ve funkci `onDisplay` použijte transformaci známou z klasického vykreslovacího řetězce.
   - Transformace se projeví na MODELVIEW matici, ze které je informace následně vyčtena a předána sestavenému programu.

Alternativně lze transformace řešit efektivně matematickou knihovnou linmath: https://github.com/datenwolf/linmath.h

Textury

- použití ve fragment shaderu pomocí globální proměnné

```
uniform sampler2D texture0; // texturovaci objekt

in vec2 uv; // texturovaci souradnice vertexu

...
void main()
{
   ...
   FragColor = texture(texture0, uv);

   ...

}
```

zabudovaná texturovací funkce GLSL

Textury

- pro propojení texturovací uniform proměnné s externím souborem stačí načíst a inicializovat texturu jako v klasickém přístupu
  - načtení např. funkcí `setTexture` z externí knihovničky `imageLoad.h`
  - nastavení parametrů:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

- textura je automaticky propojena s uniform proměnnou typu `sampler2D`
- texturovací prostředí nenastavujeme – aplikaci textury zadáváme sami v rámci fragment shaderu

Úkol 3

1. Aplikujte texturu na trojúhelník.
   - Texturu smíchejte s barvou trojúhelníku, např. násobením hodnoty z textury a barvy fragmentu.
   - Hodnotu textury `texture0` pro příslušné UV dostaneme jako `texture(texture0, uv)`.
2. Vyzkoušejte aplikaci kombinace více textur, např. vynásobením mezi sebou.

Úkol 4

1. Realizujte přepočet barvy fragmentu (`FragColor`) na základě jeho pozice (`gl_FragCoord`) a rozměru okna (`viewportDimensions`).
   - Přístup ke složkám souřadnic / rozměrů:
     - `.x`
     - `.y`

souřadnice fragmentu

$$C_{\text{new}} = C_{\text{old}} \cdot \left(2 \cdot \frac{x}{X}\right)^2$$

nová barva fragmentu

stará barva fragmentu

rozměr viewportu